

1.A Library Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Oracle** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage book loans, members, reservations, and overdue items in a municipal library.

Entities:

- Book (id, title, author, ISBN, publication_date, genre)
- Member (id, name, email, address, registration_date, status)
- Loan (id, loan_date, due_date, return_date)
- Reservation (id, reservation_date, status)
- Publisher (id, name, address, phone)
- Category (id, name, description)

Relationships:

- Member **borrows** Book (via Loan)
- Member **reserves** Book (via Reservation)
- Book **belongs to** Category
- Book **published by** Publisher
- Loan **concerns** Book and Member

1.B Library Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Cassandra** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage book loans, members, reservations, and overdue items in a municipal library.

Entities:

- Book (id, title, author, ISBN, publication_date, genre)
- Member (id, name, email, address, registration_date, status)
- Loan (id, loan_date, due_date, return_date)
- Reservation (id, reservation_date, status)
- Publisher (id, name, address, phone)
- Category (id, name, description)

Relationships:

- Member **borrows** Book (via Loan)
- Member **reserves** Book (via Reservation)
- Book **belongs to** Category
- Book **published by** Publisher
- Loan **concerns** Book and Member

1.C Library Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **BaseX** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage book loans, members, reservations, and overdue items in a municipal library.

Entities:

- Book (id, title, author, ISBN, publication_date, genre)
- Member (id, name, email, address, registration_date, status)
- Loan (id, loan_date, due_date, return_date)
- Reservation (id, reservation_date, status)
- Publisher (id, name, address, phone)
- Category (id, name, description)

Relationships:

- Member **borrow**s Book (via Loan)
- Member **reserve**s Book (via Reservation)
- Book **belongs to** Category
- Book **published by** Publisher
- Loan **concerns** Book and Member

1.D Library Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **MongoDB** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage book loans, members, reservations, and overdue items in a municipal library.

Entities:

- Book (id, title, author, ISBN, publication_date, genre)
- Member (id, name, email, address, registration_date, status)
- Loan (id, loan_date, due_date, return_date)
- Reservation (id, reservation_date, status)
- Publisher (id, name, address, phone)
- Category (id, name, description)

Relationships:

- Member **borrow**s Book (via Loan)
- Member **reserve**s Book (via Reservation)
- Book **belongs to** Category
- Book **published by** Publisher
- Loan **concerns** Book and Member

2.A Flight Reservation System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Oracle** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage flight reservations, passengers, airlines, and airports.

Entities:

- Flight (id, number, departure_date, arrival_date, status)
- Passenger (id, name, first_name, passport, email)
- Reservation (id, reservation_date, status, total_price)
- Airline (id, name, code, country)
- Airport (id, code, name, city, country)
- Seat (id, number, class, status)

Relationships:

- Passenger **reserves** Flight (via Reservation)
- Flight **departss from** Airport
- Flight **arrives at** Airport
- Flight **operated by** Airline
- Passenger **occupies** Seat (via Reservation)

2.B Flight Reservation System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Cassandra** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage flight reservations, passengers, airlines, and airports.

Entities:

- Flight (id, number, departure_date, arrival_date, status)
- Passenger (id, name, first_name, passport, email)
- Reservation (id, reservation_date, status, total_price)
- Airline (id, name, code, country)
- Airport (id, code, name, city, country)
- Seat (id, number, class, status)

Relationships:

- Passenger **reserves** Flight (via Reservation)
- Flight **departss from** Airport
- Flight **arrives at** Airport
- Flight **operated by** Airline
- Passenger **occupies** Seat (via Reservation)

2.C Flight Reservation System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **BaseX** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage flight reservations, passengers, airlines, and airports.

Entities:

- Flight (id, number, departure_date, arrival_date, status)
- Passenger (id, name, first_name, passport, email)
- Reservation (id, reservation_date, status, total_price)
- Airline (id, name, code, country)
- Airport (id, code, name, city, country)
- Seat (id, number, class, status)

Relationships:

- Passenger **reserves** Flight (via Reservation)
- Flight **departss from** Airport
- Flight **arrives at** Airport
- Flight **operated by** Airline
- Passenger **occupies** Seat (via Reservation)

2.D Flight Reservation System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **MongoDB** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage flight reservations, passengers, airlines, and airports.

Entities:

- Flight (id, number, departure_date, arrival_date, status)
- Passenger (id, name, first_name, passport, email)
- Reservation (id, reservation_date, status, total_price)
- Airline (id, name, code, country)
- Airport (id, code, name, city, country)
- Seat (id, number, class, status)

Relationships:

- Passenger **reserves** Flight (via Reservation)
- Flight **departss from** Airport
- Flight **arrives at** Airport
- Flight **operated by** Airline
- Passenger **occupies** Seat (via Reservation)

3.A Hospital Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Oracle** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage patients, doctors, appointments, hospitalizations, and invoices.

Entities:

- Patient (id, name, birth_date, phone, address)
- Doctor (id, name, specialty, badge_number)
- Appointment (id, date, time, reason, status)
- Hospitalization (id, admission_date, discharge_date, room)
- Invoice (id, date, amount, payment_status)
- Department (id, name, department_head, number_of_beds)

Relationships:

- Patient **consults** Doctor (via Appointment)
- Patient **hospitalized** in Department (via Hospitalization)
- Doctor **works** in Department
- Patient **receives** Invoice
- Appointment **concerns** Patient and Doctor

3.B Hospital Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Cassandra** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Entities:

- Patient (id, name, birth_date, phone, address)
- Doctor (id, name, specialty, badge_number)
- Appointment (id, date, time, reason, status)
- Hospitalization (id, admission_date, discharge_date, room)
- Invoice (id, date, amount, payment_status)
- Department (id, name, department_head, number_of_beds)

Relationships:

- Patient **consults** Doctor (via Appointment)
- Patient **hospitalized** in Department (via Hospitalization)
- Doctor **works** in Department
- Patient **receives** Invoice
- Appointment **concerns** Patient and Doctor

3.C Hospital Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **BaseX** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Entities:

- Patient (id, name, birth_date, phone, address)
- Doctor (id, name, specialty, badge_number)
- Appointment (id, date, time, reason, status)
- Hospitalization (id, admission_date, discharge_date, room)
- Invoice (id, date, amount, payment_status)
- Department (id, name, department_head, number_of_beds)

Relationships:

- Patient **consults** Doctor (via Appointment)
- Patient **hospitalized** in Department (via Hospitalization)
- Doctor **works** in Department
- Patient **receives** Invoice
- Appointment **concerns** Patient and Doctor

3.D Hospital Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **MongoDB** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage patients, doctors, appointments, hospitalizations, and invoices.

Entities:

- Patient (id, name, birth_date, phone, address)
- Doctor (id, name, specialty, badge_number)
- Appointment (id, date, time, reason, status)
- Hospitalization (id, admission_date, discharge_date, room)
- Invoice (id, date, amount, payment_status)
- Department (id, name, department_head, number_of_beds)

Relationships:

- Patient **consults** Doctor (via Appointment)
- Patient **hospitalized** in Department (via Hospitalization)
- Doctor **works** in Department
- Patient **receives** Invoice
- Appointment **concerns** Patient and Doctor

4.A E-Commerce System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Oracle** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage products, customers, orders, carts, and payments.

Entities:

- Product (id, name, price, stock, description)
- Customer (id, name, email, address, phone)
- Order (id, date, status, total_amount)
- Cart (id, creation_date, status)
- Payment (id, date, amount, method, status)
- Category (id, name, description)

Relationships:

- Customer **places** Order
- Customer **adds to** Cart
- Product **belongs to** Category
- Order **contains** Product
- Order **paid by** Payment

4.B E-Commerce System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Cassandra** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage products, customers, orders, carts, and payments.

Entities:

- Product (id, name, price, stock, description)
- Customer (id, name, email, address, phone)
- Order (id, date, status, total_amount)
- Cart (id, creation_date, status)
- Payment (id, date, amount, method, status)
- Category (id, name, description)

Relationships:

- Customer **places** Order
- Customer **adds to** Cart
- Product **belongs to** Category
- Order **contains** Product
- Order **paid by** Payment

4.C E-Commerce System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **BaseX** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a

piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage products, customers, orders, carts, and payments.

Entities:

- Product (id, name, price, stock, description)
- Customer (id, name, email, address, phone)
- Order (id, date, status, total_amount)
- Cart (id, creation_date, status)
- Payment (id, date, amount, method, status)
- Category (id, name, description)

Relationships:

- Customer **places** Order
- Customer **adds to** Cart
- Product **belongs to** Category
- Order **contains** Product
- Order **paid by** Payment

4.D E-Commerce System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **MongoDB** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Entities:

- Product (id, name, price, stock, description)
- Customer (id, name, email, address, phone)
- Order (id, date, status, total_amount)
- Cart (id, creation_date, status)
- Payment (id, date, amount, method, status)
- Category (id, name, description)

Relationships:

- Customer **places** Order
- Customer **adds to** Cart
- Product **belongs to** Category
- Order **contains** Product
- Order **paid by** Payment

5.A School Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Oracle** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage students, courses, teachers, grades, and enrollments.

Entities:

- Student (id, name, email, enrollment_date)
- Course (id, code, name, credits, semester)
- Professor (id, name, specialty, email)
- Enrollment (id, enrollment_date, status)
- Grade (id, type, value, date)
- Room (id, number, capacity, floor)

Relationships:

- Student **enrolls in** Course (via Enrollment)
- Professor **teaches** Course
- Course **takes place in** Room
- Student **receives** Grade
- Grade **concerns** Course and Student

5.B School Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Cassandra** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage students, courses, teachers, grades, and enrollments.

Entities:

- Student (id, name, email, enrollment_date)
- Course (id, code, name, credits, semester)
- Professor (id, name, specialty, email)
- Enrollment (id, enrollment_date, status)
- Grade (id, type, value, date)
- Room (id, number, capacity, floor)

Relationships:

- Student **enrolls in** Course (via Enrollment)
- Professor **teaches** Course
- Course **takes place in** Room
- Student **receives** Grade
- Grade **concerns** Course and Student

5.B School Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **BaseX** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage students, courses, teachers, grades, and enrollments.

Entities:

- Student (id, name, email, enrollment_date)
- Course (id, code, name, credits, semester)
- Professor (id, name, specialty, email)
- Enrollment (id, enrollment_date, status)
- Grade (id, type, value, date)
- Room (id, number, capacity, floor)

Relationships:

- Student **enrolls in** Course (via Enrollment)
- Professor **teaches** Course
- Course **takes place in** Room
- Student **receives** Grade
- Grade **concerns** Course and Student

5.D School Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **MongoDB** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage students, courses, teachers, grades, and enrollments.

Entities:

- Student (id, name, email, enrollment_date)
- Course (id, code, name, credits, semester)
- Professor (id, name, specialty, email)
- Enrollment (id, enrollment_date, status)
- Grade (id, type, value, date)
- Room (id, number, capacity, floor)

Relationships:

- Student **enrolls in** Course (via Enrollment)
- Professor **teaches** Course
- Course **takes place in** Room
- Student **receives** Grade
- Grade **concerns** Course and Student

6.A Restaurant Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Oracle** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage orders, tables, dishes, servers, and invoices.

Entities:

- Table (id, number, capacity, status)
- Customer (id, name, phone, email)
- Order (id, date, time, status)
- Dish (id, name, price, category, allergens)
- Server (id, name, badge_number, shift)
- Invoice (id, date, amount, payment_status)

Relationships:

- Customer **orders at** Table (via Order)
- Order **contains** Dish
- Server **serves** Table
- Customer **receives** Invoice
- Order **generates** Invoice

6.B Restaurant Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Cassandra** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage orders, tables, dishes, servers, and invoices.

Entities:

- Table (id, number, capacity, status)
- Customer (id, name, phone, email)
- Order (id, date, time, status)
- Dish (id, name, price, category, allergens)
- Server (id, name, badge_number, shift)
- Invoice (id, date, amount, payment_status)

Relationships:

- Customer **orders at** Table (via Order)
- Order **contains** Dish
- Server **serves** Table
- Customer **receives** Invoice
- Order **generates** Invoice

6.C Restaurant Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **BaseX** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage orders, tables, dishes, servers, and invoices.

Entities:

- Table (id, number, capacity, status)
- Customer (id, name, phone, email)
- Order (id, date, time, status)
- Dish (id, name, price, category, allergens)
- Server (id, name, badge_number, shift)
- Invoice (id, date, amount, payment_status)

Relationships:

- Customer **orders at** Table (via Order)
- Order **contains** Dish
- Server **serves** Table
- Customer **receives** Invoice
- Order **generates** Invoice

6.D Restaurant Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **MongoDB** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage orders, tables, dishes, servers, and invoices.

Entities:

- Table (id, number, capacity, status)
- Customer (id, name, phone, email)
- Order (id, date, time, status)
- Dish (id, name, price, category, allergens)
- Server (id, name, badge_number, shift)
- Invoice (id, date, amount, payment_status)

Relationships:

- Customer **orders at** Table (via Order)
- Order **contains** Dish
- Server **serves** Table
- Customer **receives** Invoice
- Order **generates** Invoice

7.A Gym Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Oracle** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage members, classes, instructors, reservations, and equipment.

Entities:

- Member (id, name, email, subscription, enrollment_date)
- Class (id, name, day, time, duration)
- Instructor (id, name, specialty, phone)
- Reservation (id, reservation_date, status)
- Equipment (id, name, type, maintenance_status)
- Room (id, name, capacity, floor)

Relationships:

- Member **reserves** Class (via Reservation)
- Instructor **leads** Class
- Class **takes place in** Room
- Member **uses** Equipment
- Equipment **located in** Room

7.B Gym Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Cassandra** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage members, classes, instructors, reservations, and equipment.

Entities:

- Member (id, name, email, subscription, enrollment_date)
- Class (id, name, day, time, duration)
- Instructor (id, name, specialty, phone)
- Reservation (id, reservation_date, status)
- Equipment (id, name, type, maintenance_status)
- Room (id, name, capacity, floor)

Relationships:

- Member **reserves** Class (via Reservation)
- Instructor **leads** Class
- Class **takes place in** Room
- Member **uses** Equipment
- Equipment **located in** Room

7.C Gym Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **BaseX** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage members, classes, instructors, reservations, and equipment.

Entities:

- Member (id, name, email, subscription, enrollment_date)
- Class (id, name, day, time, duration)
- Instructor (id, name, specialty, phone)
- Reservation (id, reservation_date, status)
- Equipment (id, name, type, maintenance_status)
- Room (id, name, capacity, floor)

Relationships:

- Member **reserves** Class (via Reservation)
- Instructor **leads** Class
- Class **takes place in** Room
- Member **uses** Equipment
- Equipment **located in** Room

7.D Gym Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **MongoDB** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage members, classes, instructors, reservations, and equipment.

Entities:

- Member (id, name, email, subscription, enrollment_date)
- Class (id, name, day, time, duration)
- Instructor (id, name, specialty, phone)
- Reservation (id, reservation_date, status)
- Equipment (id, name, type, maintenance_status)
- Room (id, name, capacity, floor)

Relationships:

- Member **reserves** Class (via Reservation)
- Instructor **leads** Class
- Class **takes place in** Room
- Member **uses** Equipment
- Equipment **located in** Room

8.A Parking Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Oracle** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage parking spaces, vehicles, entries/exits, and payments.

Entities:

- Space (id, number, type, status)
- Vehicle (id, license_plate, type, brand)
- EntryExit (id, date_time, type, duration)
- Payment (id, date, amount, method)
- Customer (id, name, phone, email)
- Zone (id, name, hourly_rate, capacity)

Relationships:

- Vehicle **parked in** Space
- Customer **uses** Space (via EntryExit)
- EntryExit **concerns** Vehicle and Space
- Customer **pays** EntryExit (via Payment)
- Space **located in** Zone

8.C Parking Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **Cassandra** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage parking spaces, vehicles, entries/exits, and payments.

Entities:

- Space (id, number, type, status)
- Vehicle (id, license_plate, type, brand)
- EntryExit (id, date_time, type, duration)
- Payment (id, date, amount, method)
- Customer (id, name, phone, email)
- Zone (id, name, hourly_rate, capacity)

Relationships:

- Vehicle **parked in** Space
- Customer **uses** Space (via EntryExit)
- EntryExit **concerns** Vehicle and Space
- Customer **pays** EntryExit (via Payment)
- Space **located in** Zone

8.C Parking Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **BaseX** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage parking spaces, vehicles, entries/exits, and payments.

Entities:

- Space (id, number, type, status)
- Vehicle (id, license_plate, type, brand)
- EntryExit (id, date_time, type, duration)
- Payment (id, date, amount, method)
- Customer (id, name, phone, email)
- Zone (id, name, hourly_rate, capacity)

Relationships:

- Vehicle **parked in** Space
- Customer **uses** Space (via EntryExit)
- EntryExit **concerns** Vehicle and Space
- Customer **pays** EntryExit (via Payment)
- Space **located in** Zone

8.D Parking Management System

Dato il seguente schema entità/relazione, fare un confronto tra **Neo4** e **MongoDB** considerando lo stesso dataset di dimensione crescente e 4 query di complessità crescente. Per far ciò, scegliere un linguaggio di programmazione a piacere al fine di interagire con i due database per svolgere le operazioni di inserimento dati e di esecuzione delle query. Gli esperimenti dovranno essere automatizzati.

Dimostrare quale database a parità di condizioni hardware/software presenta tempi di esecuzione delle query migliori. Gli schemi entità/relazione sono indicativi e possono essere riadattati in base a motivate esigenze progettuali.

Seguire la guida alla scrittura della tesina.

Problem: Manage parking spaces, vehicles, entries/exits, and payments.

Entities:

- Space (id, number, type, status)
- Vehicle (id, license_plate, type, brand)
- EntryExit (id, date_time, type, duration)
- Payment (id, date, amount, method)
- Customer (id, name, phone, email)
- Zone (id, name, hourly_rate, capacity)

Relationships:

- Vehicle **parked in** Space
- Customer **uses** Space (via EntryExit)
- EntryExit **concerns** Vehicle and Space
- Customer **pays** EntryExit (via Payment)
- Space **located in** Zone

